

Reviewer Pack — Generated Algebra Dimension of Layer Operators Bounds IP_2 R...

Entry 59a86c7a7d5c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-18 07:28:12 UTC

Generated Algebra Dimension of Layer Operators Bounds IP_2 Read-Twice BP

Entry ID: 59a86c7a7d5c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-18 07:28:12 UTC

1. Conjecture

Field A (mathematical branch): Operator algebra / Burnside-Schur-Wedderburn theory — the unital R -subalgebra $A(P) \subseteq R^{\{w \times w\}}$ generated by the layer transition matrices $\{T_i^{\{b\}}\}$ of a branching program P , with $\dim A(P)$ computed by iterative span closure (start from identity and all $T_i^{\{b\}}$ flattened to $R^{\{w^2\}}$; multiply each existing basis element by each layer operator, add the product to the span via Gauss-Jordan, iterate until no new dimension is found). Distinct from prior free-probability attempts (S-transform, Laplacian R -transform, free cumulants on Max-Cut) which targeted communication / SDP gaps; here the algebra invariant is a STRUCTURAL property of the BP wiring (not its truth table — two BPs computing the same function can have very different $\dim A$), shielding from Razborov-Rudich. The closure uses real linear-algebra rank operations (not ring identities over F_q), shielding from Aaronson-Wigderson algebrization. An arXiv search ‘generated matrix algebra’ AND (‘branching program’ OR ‘read-twice’ OR ‘Borodin Razborov Smolensky’) returns 0 direct hits with <5 adjacent papers (all on Barrington permutation BPs over a fixed simple group algebra, never on the generated algebra of arbitrary read-twice transitions). **Field B** (complexity object): Read-twice oblivious BP size $s(P)$ for $IP_2(x,y) = \text{XOR}_k (x_k \text{ AND } y_k)$ on $2n$ inputs under the adversarial variable order $x_1, \dots, x_n$ followed by $y_1, \dots, y_n$ (Borodin-Razborov-Smolensky 1993 regime); instantiating the focus’s ‘distinguishing tensor’ formalism: find $\rho(P)$ with $\rho(P) = O(\log \text{size}(P))$ but $\rho(P_{IP}) = \Omega(n)$ for any P computing IP_2 .

Statement:

Let P be a read-twice oblivious BP on $2n$ inputs with width w , length L , and $2L$ layer transition matrices $\{T_i^{\{b\}} \in \{0,1\}^{\{w \times w\}} : i \in [L], b \in \{0,1\}\}$ where each row of $T_i^{\{b\}}$ has at most one 1. Let $A(P) \subseteq R^{\{w \times w\}}$ be the unital R -subalgebra generated by the identity and the $\{T_i^{\{b\}}\}$, and define $\rho(P) := \log_2(\dim_R A(P) + 1)$. Conjecture: (i) for every read-twice oblivious BP P , $\rho(P) \leq 2 \cdot \log_2(w + 1)$ (trivially since $\dim A(P) \leq w^2 \leq \text{size}(P)^2$); (ii) for every read-twice oblivious BP P computing IP_2 under the adversarial input order, $\rho(P) \geq n - 2$. A single read-twice oblivious BP P with $f_P = IP_2$ (adversarial order) and $\rho(P) < n - 2$ refutes (ii); a single BP violating (i) refutes (i).

Rationale (proposer’s reasoning):

$A(P)$ encodes EVERY product of layer transitions, i.e. all reachable state-permuters of the BP’s syntactic dynamical system — a relational/categorical span, not a single truth-table number, so it dodges the textbook Razborov-Rudich trap and the recent calibration warnings against truth-table single-number invariants. By the BRS rectangle argument any

read-once BP for IP_2 in adversarial order must distinguish 2^n x-prefixes at the midpoint cut; we conjecture this distinguishing power forces a Schur-Wedderburn-large generated algebra even with two reads per variable, because the y-phase must non-trivially recombine all 2^n prefix-states into a single parity bit. The span-closure construction uses real rank operations only, so it does not extend to a polynomial F_q relaxation and is shielded from Aaronson-Wigderson algebrization.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6f9b8b77eef5d178

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 360 random read-twice BPs ($n \in \{3..8\}$, $w \in \{4,8\}$, 30 seeds each), $\rho(P) \leq 2 \cdot \log_2(w+1)$ must hold for every instance (clause i). For the 3 canonical adversarial IP_2 BPs ($n \in \{2,3,4\}$), $\rho(P) \geq n-2$ must hold for every instance (clause ii). For friendly-order IP_2 BPs ($n \in \{2..8\}$), $\rho(P) \leq 2$ for every instance.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.85	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - generated matrix algebra branching program lower bound - read-twice oblivious branching program IP_2 inner product Borodin Razborov Smolensky - transition matrix algebra dimension branching program width

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random
import math
import json
from fractions import Fraction

def matrix_mult(a, b):
    """Multiply two matrices a and b."""
```

```

    return [[sum(a[i][k] * b[k][j] for k in range(len(b))) for j in range(len(b[0]))] for i in range(len(a))]

def matrix_add(a, b):
    """Add two matrices a and b."""
    return [[a[i][j] + b[i][j] for j in range(len(a[0]))] for i in range(len(a))]

def matrix_to_vector(m):
    """Convert a matrix to a vector by flattening it."""
    return [m[i][j] for i in range(len(m)) for j in range(len(m[0]))]

def vector_to_matrix(v, w):
    """Convert a vector to a matrix of size w x w."""
    return [[v[i * w + j] for j in range(w)] for i in range(w)]

def gaussian_elimination(matrix):
    """Perform Gaussian elimination on a matrix."""
    n = len(matrix)
    for i in range(n):
        # Find the row with the maximum element in the current column
        max_row = i
        for k in range(i + 1, n):
            if abs(matrix[k][i]) > abs(matrix[max_row][i]):
                max_row = k
        # Swap the current row with the max_row
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        # If the leading element is zero, the matrix is singular
        if matrix[i][i] == 0:
            continue
        # Normalize the current row
        for k in range(i + 1, n):
            factor = Fraction(matrix[k][i], matrix[i][i])
            for j in range(i, n):
                matrix[k][j] -= factor * matrix[i][j]
    return matrix

def compute_dimension(basis):
    """Compute the dimension of the space spanned by the basis vectors."""
    if not basis:
        return 0
    # Perform Gaussian elimination on the basis vectors
    basis = gaussian_elimination(basis)
    # Count the number of non-zero rows
    dimension = 0
    for row in basis:
        if any(row):
            dimension += 1
    return dimension

def generate_random_bp(n, w, seed):
    """Generate a random read-twice oblivious BP."""
    random.seed(seed)
    L = 4 * n
    # Assign each variable to exactly two random layer positions
    var_positions = {}
    for k in range(n):
        positions = random.sample(range(L), 2)

```

```

        var_positions[k] = positions
# Generate random transition matrices
T = []
for i in range(L):
    T_i = []
    for b in [0, 1]:
        # Each row of  $T_i^{(b)}$  has at most one 1
        T_ib = [[0] * w for _ in range(w)]
        for j in range(w):
            if random.random() < 0.5:
                k = random.randint(0, w - 1)
                T_ib[j][k] = 1
        T_i.append(T_ib)
    T.append(T_i)
return T, var_positions

def compute_rho(P, var_positions, w):
    """Compute  $\rho(P)$  for a given BP  $P$ ."""
    # Initialize the basis with the identity matrix
    basis = [matrix_to_vector([[1 if i == j else 0 for j in range(w)] for i in range(w)])]
    # Generate all possible products of transition matrices
    for k in range(len(var_positions)):
        positions = var_positions[k]
        for b in [0, 1]:
            # Multiply the transition matrix  $T_i^{(b)}$  with the current basis
            new_basis = []
            for v in basis:
                m = vector_to_matrix(v, w)
                T_ib = P[positions[0]][b] if b == 0 else P[positions[1]][b]
                m_new = matrix_mult(m, T_ib)
                new_basis.append(matrix_to_vector(m_new))
            # Add the new basis vectors to the basis
            basis.extend(new_basis)
    # Compute the dimension of the space spanned by the basis vectors
    dimension = compute_dimension(basis)
    # Compute  $\rho(P)$ 
    rho = math.log2(dimension + 1)
    return rho

def run_trial(seed):
    """Run a single trial with the given seed."""
    random.seed(seed)
    n = random.choice([3, 4, 5, 6, 7, 8])
    w = random.choice([4, 8])
    # Generate a random read-twice oblivious BP
    P, var_positions = generate_random_bp(n, w, seed)
    # Compute  $\rho(P)$ 
    rho = compute_rho(P, var_positions, w)
    # Check the conjecture
    conjecture_holds = rho <= 2 * math.log2(w + 1)
    counterexample = "" if conjecture_holds else f"rho={rho} > 2*log2(w+1)={2*math.log2(w+1)}"
    return {
        "metric_name": "rho",
        "metric_value": rho,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
    }

```

```

        "counterexample": counterexample
    }

def main():
    """Main function to run the trials."""
    seeds = sys.argv[1:] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps({'seed': seed, **result})}")
        results.append(result)
    # Compute statistics
    metric_values = [r["metric_value"] for r in results]
    mean = sum(metric_values) / len(metric_values)
    std = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)
    # Determine the result
    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    else:
        for r in results:
            if not r["conjecture_holds"]:
                print(f"RESULT: FALSIFIED counterexample={r['counterexample']} first_failing_seed={seed}")
                break

if __name__ == "__main__":
    main()

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e6d0879e.py", line 158, in <module>
main()

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e6d0879e.py", line 140, in main
result = run_trial(seed)

^^^^^^^^^^^^^^^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e6d0879e.py", line 123, in run_trial
rho = compute_rho(P, var_positions, w)

^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e6d0879e.py", line 110, in compute_rho
dimension = compute_dimension(basis)

^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e6d0879e.py", line 60, in compute_dimension
basis = gaussian_elimination(basis)

^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e6d0879e.py", line 52, in gaussian_elimination
matrix[k][j] -= factor * matrix[i][j]

~~~~~^^^

IndexError: list index out of range

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test crashed with an `IndexError` inside `gaussian_elimination` before producing any `RESULT` line or data, so neither the support nor falsification condition can be evaluated.  
| next: Fix the Gaussian elimination routine (ensure consistent row/column dimensions and bounds checks) and rerun the 360 random + canonical/friendly IP\_2 trials.

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider   | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|------------|------------------|-----------|-----|--------------|
| 1 | propose         | claude_max | opus             | 0         | 0   | 301675       |
| 2 | preregistration | claude_max | opus             | 0         | 0   | 6925         |
| 3 | novelty         | claude_max | opus             | 0         | 0   | 4439         |
| 4 | novelty         | claude_max | opus             | 0         | 0   | 8129         |
| 5 | test_gen        | mistral    | codestral-latest | 0         | 0   | 314522       |
| 6 | test_gen        | mistral    | codestral-latest | 0         | 0   | 314535       |
| 7 | test_gen        | mistral    | codestral-latest | 0         | 0   | 316239       |
| 8 | test_gen        | mistral    | codestral-latest | 0         | 0   | 311667       |
| 9 | judge           | claude_max | opus             | 0         | 0   | 9369         |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1587499 ms total latency. Provider mix: {'claude\_max': 5, 'mistral': 4}

*(full prompt+response transcripts available in `research/audit/59a86c7a7d5c.jsonl`)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/59a86c7a7d5c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/59a86c7a7d5c.tar.gz` (if generated)